Advanced Memory Attacks

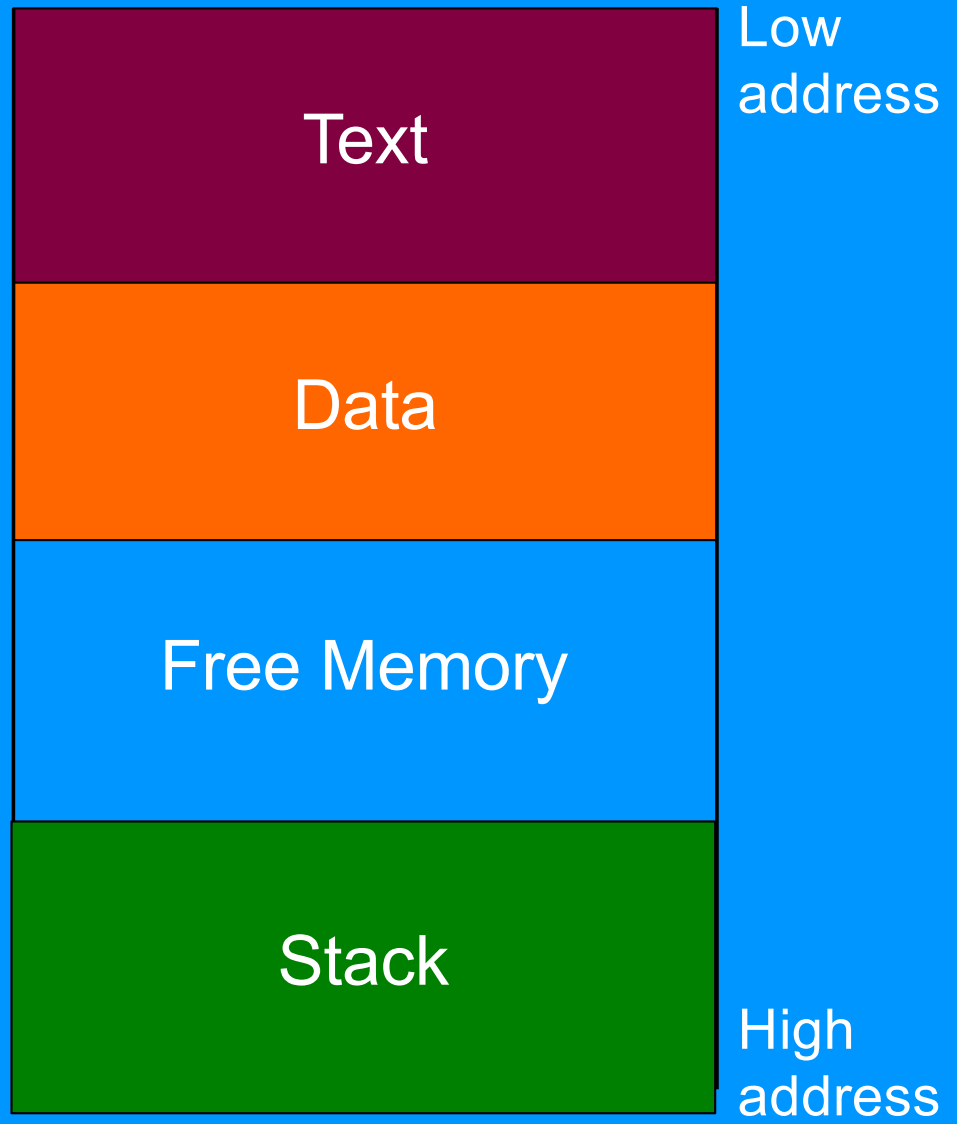
Tom Chothia

What I want you to understand …

- What are return to libc attacks.
- What are integer overflow attacks.
- What are use after free attacks.
- What are format string attacks.
- To be able to spot and exploit very simple examples of these.

Defense. The NX-bit: W xor X.

- Code should be in the text area of the memory. Not on the stack.
- The NX-bit provides a hardware distinction between the text and stack.
- When enabled, the program will crash if the EIP ever points to the stack.



Code Reuse

- We bypass W xor X protection by reusing code from the execute section of the memory.
- This can be functions from the main binary
- Or from any loaded libraries.
- We can also jump to any point inside a function, not just the start.

Return to Libc

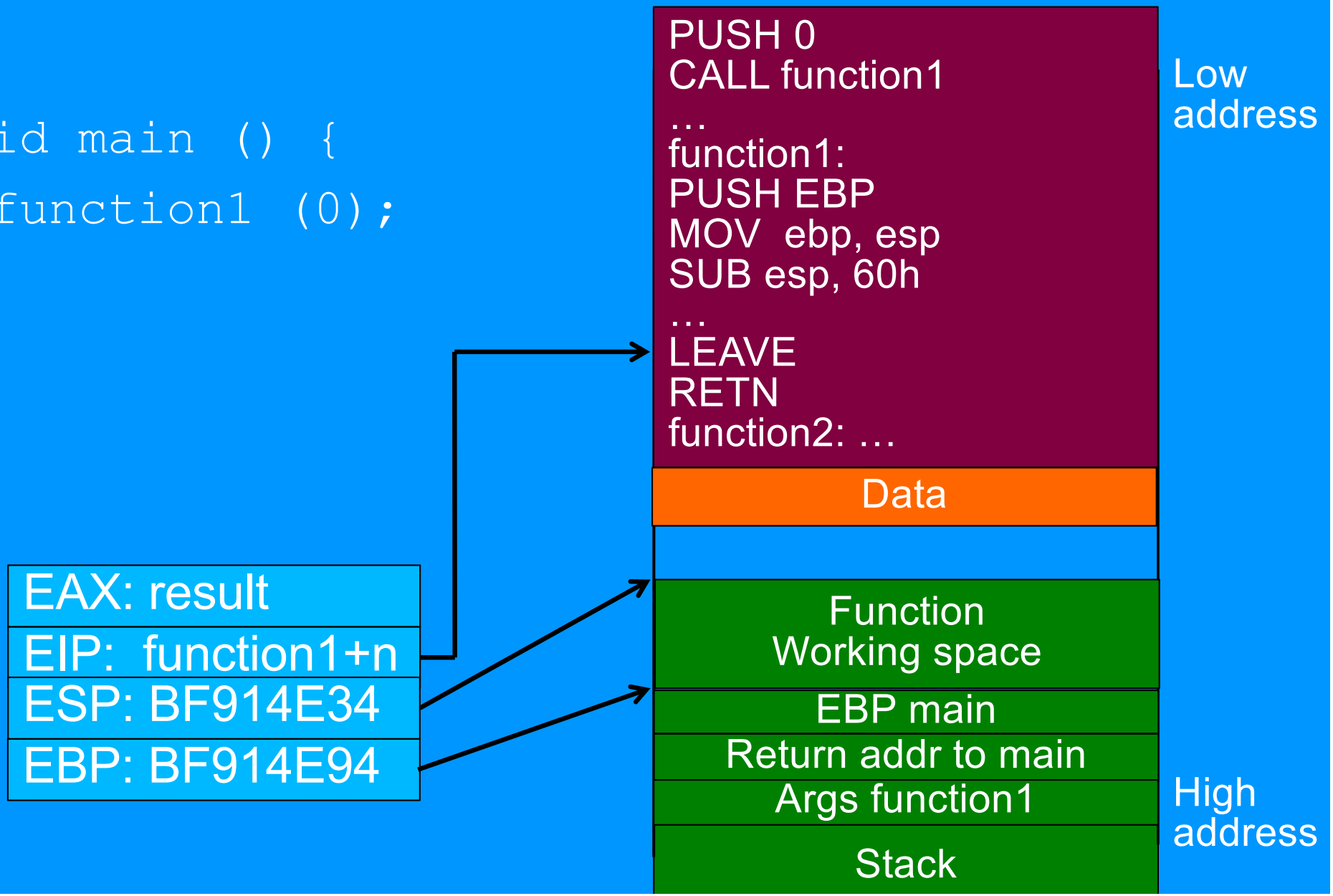
- The standard C library ”libc” is almost always loaded.
- This has lots of useful functions we could call
- Like “system” that will run any command.
- The memory map tells you where it is loaded.
  - /proc/<process ID>/maps for running Linux processes.
- Function offset can be found in IDA (or many other tools, readelf, gdb …)

Why did our Return to Libc program crash?

- Our return to libc injection crashed the process.
- This might not matter, but:
  - It might alert a sys admin to an attack
  - It might bring down a remove server that we want to keep up for repeated access.
- Why did it crash?

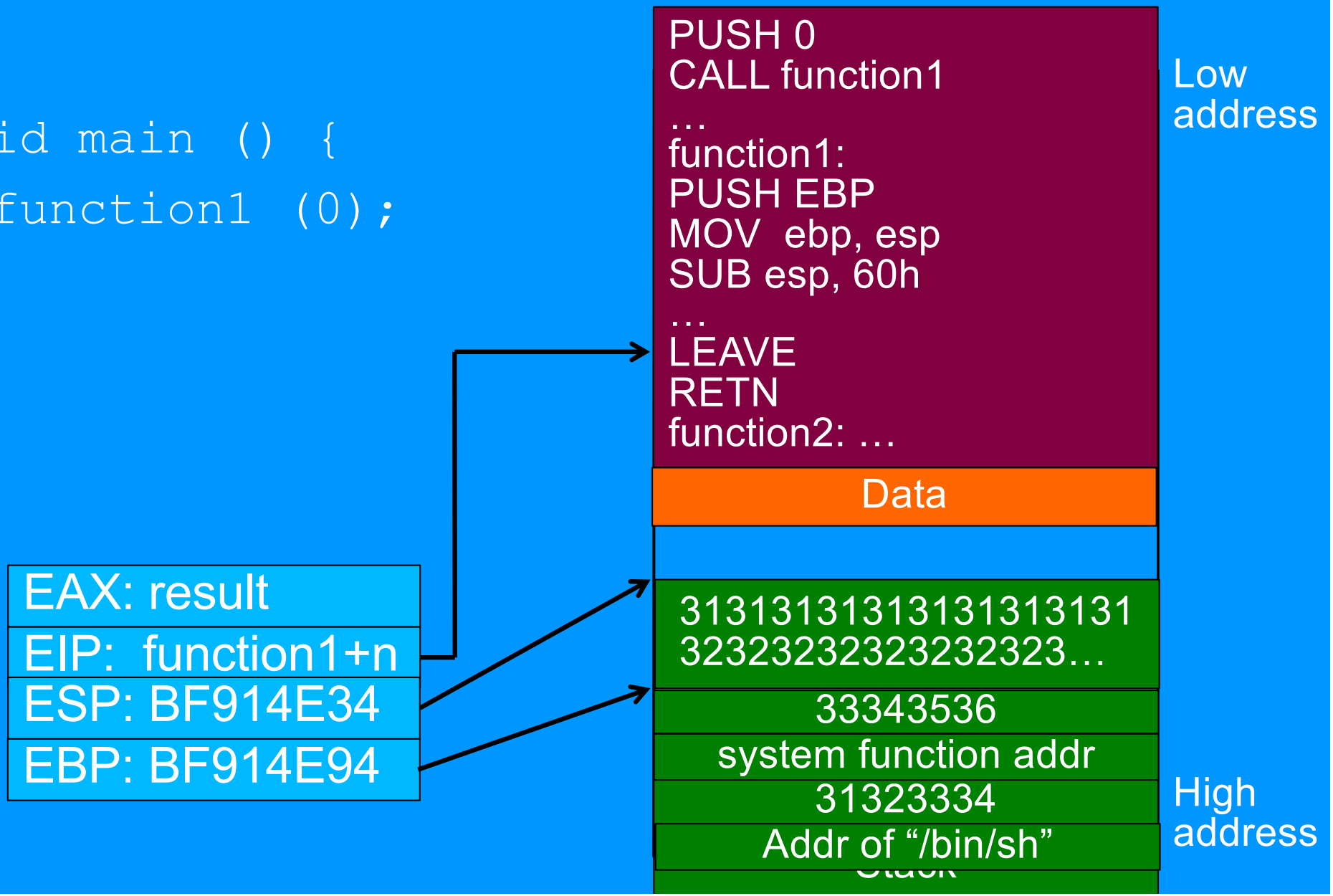
A Normal Run

```
void main () {
   function1 (0);
}
```

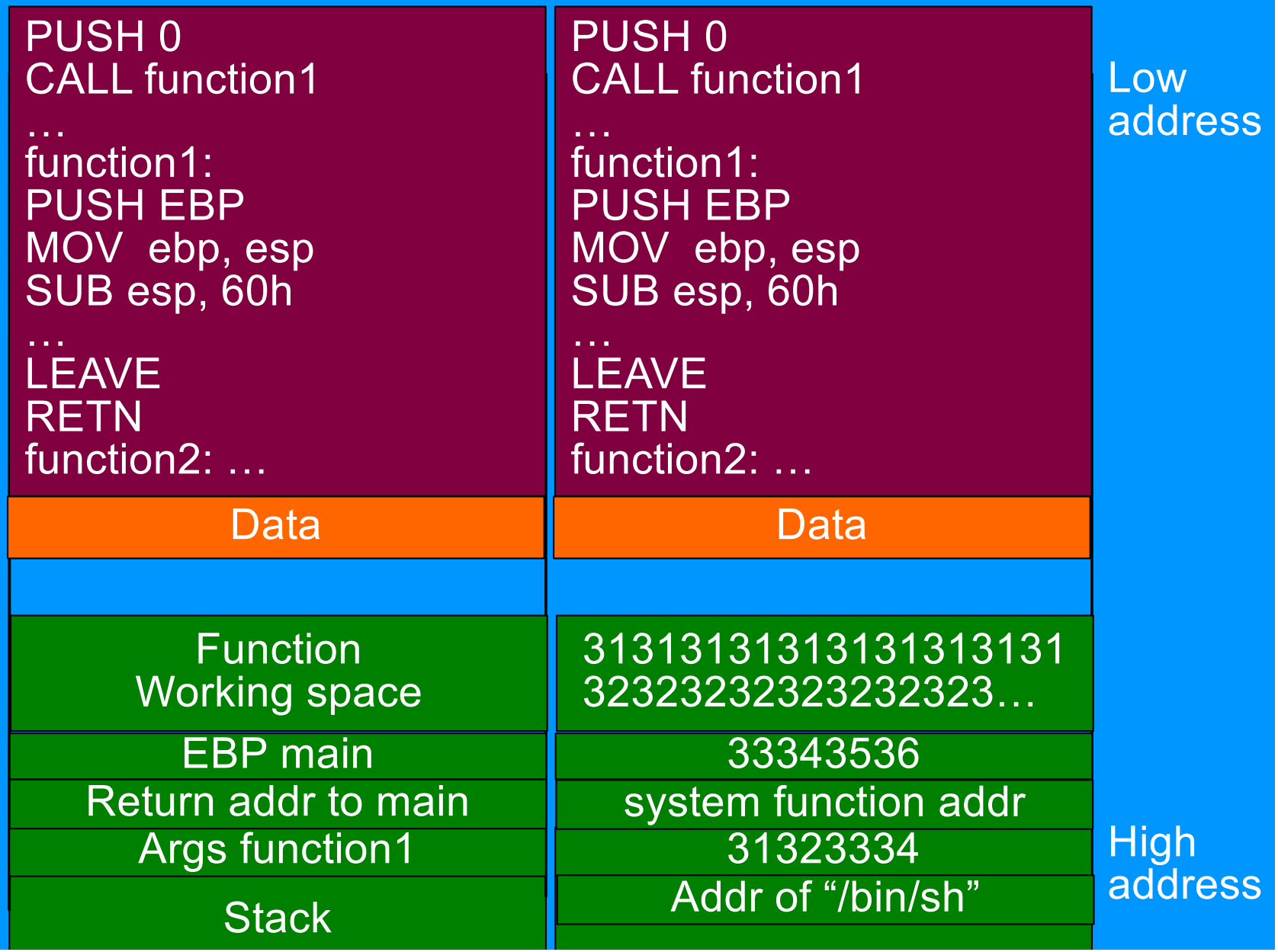


During our attack:

```
void main () {
   function1 (0);
}
```

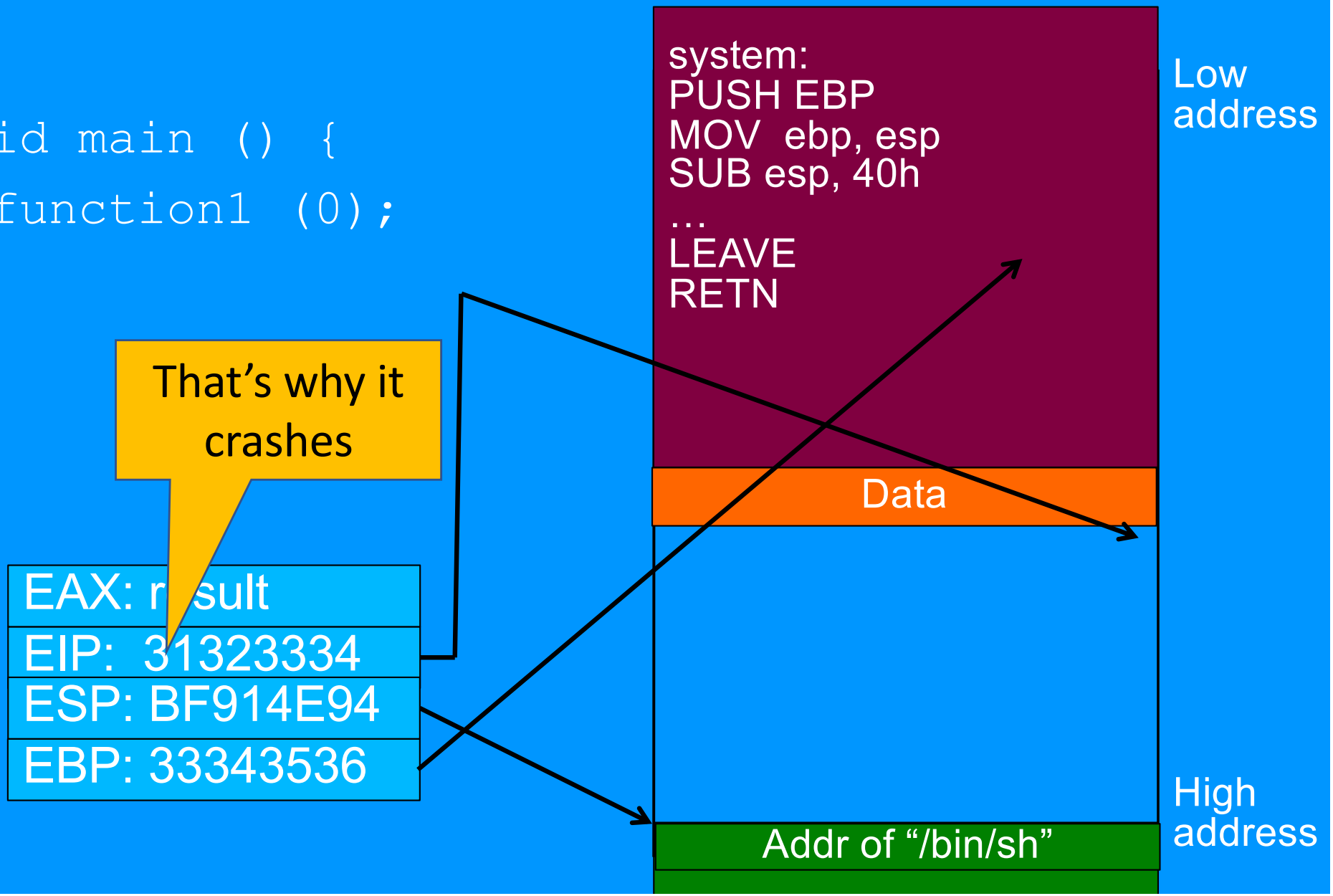


During our attack:

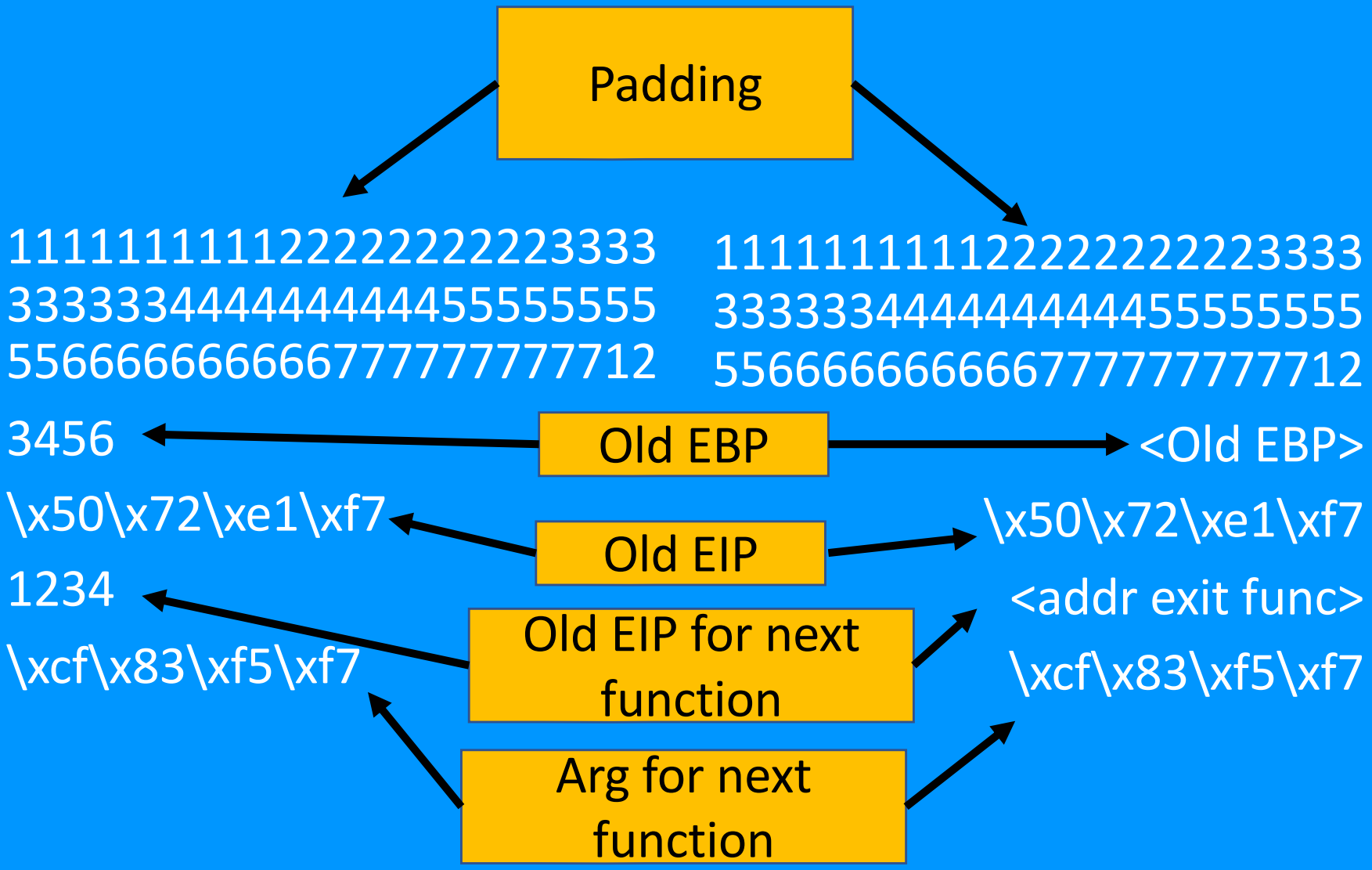


During our attack:

```
void main () {
   function1 (0);
}
```



The attack string:



Key Idea

- We can queue up instruction pointers on the stack and execute one after another.
- We can use this to run any number of code segments in any order.
- This makes code reuse powerful and flexable.

Powerful and Flexible Attack

The Geometry of Innocent Flesh on the Bone: Return-into-libc without Function Calls (on the x86)

Hovav Shacham*
Department of Computer Science & Engineering
University of California, San Diego
La Jolla, California, USA
hovav@hovav.net

ABSTRACT

We present new techniques that allow a return-into-libc attack to be mounted on x86 executables that calls *no functions at all*. Our attack combines a large number of short instruction sequences to build *gadgets* that allow arbitrary computation. We show how to discover such instruction sequences by means of static analysis. We make use, in an essential way, of the properties of the x86 instruction set.

Categories and Subject Descriptors

D.4.6 [**Operating Systems**]: Security and Protection

General Terms

Security, Algorithms

Keywords

Return-into-libc, Turing completeness, instruction set

1. INTRODUCTION

We present new techniques that allow a return-into-libc

using the short sequences we find in a specific distribution of GNU libc, and we conjecture that, because of the properties of the x86 instruction set, in any sufficiently large body of x86 executable code there will feature sequences that allow the construction of similar gadgets. (This claim is our *thesis*.) Our paper makes three major contributions:

1. We describe an efficient algorithm for analyzing libc to recover the instruction sequences that can be used in our attack.

2. Using sequences recovered from a particular version of GNU libc, we describe gadgets that allow arbitrary computation, introducing many techniques that lay the foundation for what we call, facetiously, *return-oriented programming*.

3. In doing the above, we provide strong evidence for our thesis and a template for how one might explore other systems to determine whether they provide further support.

In addition, our paper makes several smaller contributions. We implement a return-oriented shellcode and show how it

Integer Overflow

- Data types matter!
- When you run out of bits you loop back to zero.
- This can lead to bypassing protections and bad memory read and writes.

Type	Storage size	Value range
char	1 byte	-128 to 127 or 0 to 255
unsigned char	1 byte	0 to 255
signed char	1 byte	-128 to 127
int	2 or 4 bytes	-32,768 to 32,767 or -2,147,483,648 to 2,147,483,647
unsigned int	2 or 4 bytes	0 to 65,535 or 0 to 4,294,967,295
short	2 bytes	-32,768 to 32,767
unsigned short	2 bytes	0 to 65,535
long	8 bytes	-9223372036854775808 to 9223372036854775807
unsigned long	8 bytes	0 to 18446744073709551615

Integer Overflow attacks

- Perfect example of what this module is about.
- You might think “int” behaves like the numbers you learn in maths. **It doesn’t.**
- If you understand how these things really work, you can control them and exploit the system.
- Cyber security attacks happen when people’s assumptions about how their system works are 99.9% correct.

Apple Zero-click iMessage Exploit

- Take over iPhone just by sending victim a single iMessage.
- Developed by NSO Group for use by sketchy governments everywhere.
- Complex attack, but partly based on an integer overflow.
- The size of the message font could be set to a huge value, leading to an arbitrary write to memory.
- Full details: https://googleprojectzero.blogspot.com/2021/12/a-deep-dive-into-nso-zero-click.html

Memory allocation vulnerabilities.

- Memory management in C can be hard!
- If we copy to (or read from) allocated memory with the incorrect bound, we can overflow (or learn) heap memory.
- Incorrect allocation can also lead to vulnerabilities.
- We want: malloc -> use -> free.
- malloc -> use ~~-> free~~ = memory leak
- malloc -> use -> free -> use = ”Use after free”.

Use after free: malloc -> use -> free -> use

- After `free` the data remains in memory so the program may work.
- However other data may be allocated in its place.
- If the attacker controls the new data, they also control the old data

```
int * importantSecInfo = malloc(size of int);
free(importantSecInfo);
int * attackerControlledInput = malloc(size of int);
if(condition(importantSecInfo)) {do something important}
```

Double free: malloc -> use -> free -> free

```
int * anyData = malloc(size of int);
…
free(anyData);
…
free(anyData);
…
int * importantSecInfo = malloc(size of int);
…
int * attackerControlledInput = malloc(size of int);
```

Now `importantSecInfo` and `attackerControlledInput` might point to the say section of memory

Giving the attack control of `importantSecInfo`

Format String Vulnerabilities

- int printf(const char *format, ...)
- printf(“user: %s has %d cash”, username, money)
- printf(“string value %s as address %p”, username, username)

There is no check that the number of % in the string, matches the number of arguments.

What would printf(“value: %p”) print???

Format String Vulnerabilities

- printf(“value: %p”)
- Code for printf will reach the “%p” and then will look for the argument.
- The first argument “value: %p” will be in RSI register, so the code will look in RDI and print whatever is there.
- printf(%p %p %p %p %p %p %p %p) will print RSI, RDX, RCX, R8, R9 and the top three values on the stack.
- If the attackers can control the string being printed, then they can look on the stack find addresses, and the stack canary

Summary

- Modern memory defenses make attacks much harder.
- If is not enough just to spot a gets, indeed the originally namecheck.c program is secure if compiled normally.
- Attack usually improve a combination of the techniques and/or a lot of luck.
- Turning a seg fault into code execution can takes months and a team of experts.

What I want you to understand …

- What are return to libc attacks.
- What are integer overflow attacks.
- What are use after free attacks.
- What are format string attacks.
- To be able to spot and exploit very simple examples of these.